

Why Programming Still Matters in the Age of AI

Computer Science 2 — before we open the syllabus

Sumin Han

`suminhan@ksa.hs.kr`

Let's Ask It Properly

The question you are already thinking

"AI writes code now. It writes it faster than me, and honestly **better** than me. So why am I sitting in a programming class?"

That is a **fair question**, and I am not going to answer it by telling you AI is bad at coding.

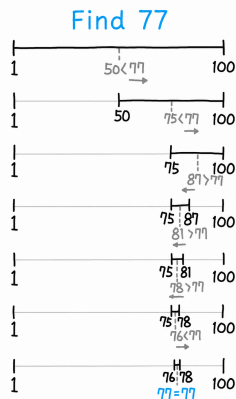
It isn't. It's very good, and it is getting better every month.

I am going to answer it a different way: by showing you **what job just opened up** because of that.

Yes — This Part Is Really True

Ask for a binary search, and you get one. Instantly. Correct.

```
def binary_search(L, target):
    lo, hi = 0, len(L) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if L[mid] == target: return mid
        elif L[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```



Ten years ago this was a week of the course. Now it is four seconds. So **what got expensive?**

The Bottleneck Moved

Before

Hard part: **writing** the code.

You wrote 100 lines a day, and you understood all 100 — because you had to, to write them.

Now

Hard part: **judging** the code.

You can generate 1000 lines in a minute. **Do you understand any of them?**

**AI improving does not shrink this job.
It grows it.**

This Is Already Showing Up — 2026

The first step is squeezed

Harder for new grads and interns to get hired.

Average seniority at big tech companies **keeps rising**.

But demand is not shrinking

Top tech companies are hiring **about 20% more** than a year ago.

AI engineering listings: **50–100% more** at Apple, Google, etc.

Source: *The Pragmatic Engineer*, "State of the Software Engineering Job Market" (2026, TrueUp hiring data) — US market.

Both are true at once. **Why?**

The way in used to be *"here is a small, well-defined task — go implement it."* **That is exactly what AI does best now. That first step is gone.**

What is left starts one level up: decide, direct, verify.

“But I Can Just Ask in Plain Language”

True. You can prompt in whatever language you think in — and you should.

But look at which direction that helps

Going *in*, you get to use human language.

Coming *back*, **the answer is always code**. There is no human-language version of the output. There is only the code, and whether you can read it.

**If you cannot read what comes back,
you did not give an instruction —
you made a wish.**

“Write Me a Grade Calculator”

So I asked for one. Here is what came back — clean, readable, runs fine:

```
def final_grade(scores, attendance_rate):  
    average = sum(scores) / len(scores)  
    if attendance_rate < 0.75:  
        return "Fail"  
    return "Pass" if average >= 70 else "Fail"
```

Looks right. Runs. No errors. **And it is wrong for this course.**

Here is the part that matters: nothing about it looks wrong. It will not warn you. It just quietly computes the wrong grade.

Three Things It Could Not Have Known

- A student misses the exam for a **medical reason**. Our rule gives a make-up score weighted at **80%** — and **100%** for official absence. The code has neither.
- A student **failed and took the re-test**. Which score counts? The code has never heard of a re-test.
- Attendance below $3/4$ — does that override everything, or is it checked first? The code guessed. **It guessed differently than we do.**

“Then just put the rules in the prompt!” — sure. **But you only know to do that if you already noticed they were missing.** And the code never told you. **You had to read it to see the holes.**

“I Tested It. It Works.”

Task: return True if a year is a leap year.

```
def is_leap_year(year):  
    if year % 4 == 0:  
        return True  
    else:  
        return False
```

(% gives the remainder — `year % 4 == 0` means “divides evenly by 4”.)

You test it:

`is_leap_year(2024)`

⇒ True correct!

It passed. Ship it?

Now Try This One

```
is_leap_year(1900)
```

Expected: False

1900 was not a leap year.

Execution Result

True

Why: the real rule has three parts — divisible by **4**, *except* centuries, *unless* divisible by **400**. The code only knows the first part. 1900 divides by 100 but not 400, so it is **not** a leap year.

The part that should worry you

The bug was always there. Your test just never looked in the right place.
Passing a test is not the same as being right.

Choosing the input that breaks it — that was the skill. And no one could do it for you.

September 1999: Ten Months, Then Silence

NASA's **Mars Climate Orbiter** flew for ten months. On arrival it was supposed to settle into orbit about **150 km** above Mars.

It came in at **57 km** — deep into the atmosphere. It broke apart. **Nobody ever heard from it again.**

What went wrong

One team wrote the thruster numbers in **pounds of force**. The other team read them as **newtons**.

1 lbf = 4.45 N, so every course correction was off **by a factor of 4.45** — for ten months. Cost: **\$125 million**.

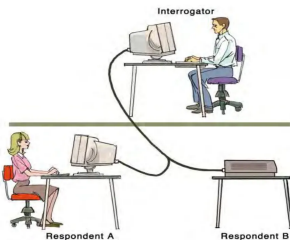
Every line of that code ran perfectly. Nobody checked what the number meant.

1950: The Chair Turing Left Empty



Alan Turing, 1936.

The Father of the Computer.



The Imitation Game (1950).

Turing asked “*Can machines think?*” — and refused to answer it. He turned it into a test instead: a **human judge (the interrogator)** questions both sides and decides whether the answers hold up.

At the very beginning of computing, the design already had **a person in the chair, asking the questions**. **That chair never got automated.**

This Is What the Syllabus Actually Asks of You

The four Subject Core Competencies for this course — and where you just met each one:

- **Fundamental Programming & Problem-Solving**
Read what comes back and understand what it really does.
- **Logical Thinking & Analytical Skills**
Know which input would break it — before it breaks.
- **Communication & Collaboration**
Why we work problems *together*, out loud, in class.
- **Ethical Use of AI & Responsible Attitude**
Use the tool, then **own the answer**. “The AI wrote it” is not a defence.

So — Why Are You Here?

- **Not** to type code faster than a machine.
- To be the person who can say exactly what is wanted, read what comes back, and know which input would break it.

So here is how this class runs

In class we solve problems together, by hand, without AI. Not because using AI is cheating — but because **you cannot build a skill that the machine performs for you.**

**You will use a calculator your whole life.
You still learned arithmetic by hand.**